

Polynomial regression:

Method 1 : Compare Linear vs Polynomial

Import all necessary Libraries:

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

dataset = pd.read_csv(r"C:\Users\Lenovo\Downloads\emp_sal.csv")
```

```
In [6]: X = dataset.iloc[:,1:2].values
y = dataset.iloc[:,2].values

from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=4)

X_poly = poly.fit_transform(X)

lin_reg2 = LinearRegression()
lin_reg2.fit(X_poly, y)

X_grid = np.arange(X.min(), X.max(), 0.1)
X_grid = X_grid.reshape(-1,1)
```

```
In [7]: plt.figure(figsize=(8,5))

plt.scatter(X,
            y,
            color='black',
            label='Actual Data')

plt.plot(X,
         lin_reg.predict(X),
         color='blue',
         linewidth=2,
         label='Linear Regression')

plt.plot(X_grid,
         lin_reg2.predict(poly.transform(X_grid)),
         color='red',
         linewidth=3,
```

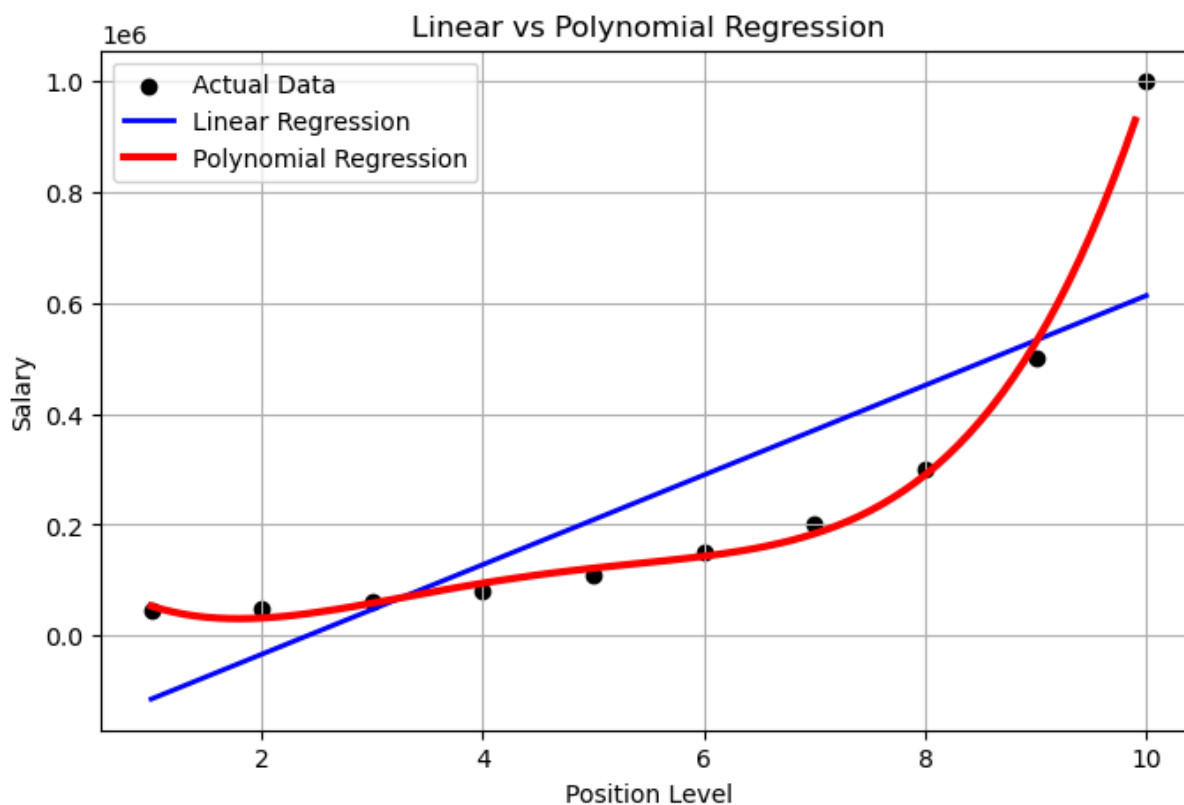
```

label='Polynomial Regression')

plt.title("Linear vs Polynomial Regression")
plt.xlabel("Position Level")
plt.ylabel("Salary")
plt.legend()
plt.grid(True)

plt.show()

```



Method 2 : Show Preicted points:

```

In [8]: prediction = lin_reg.predict(X)

plt.figure(figsize=(6,4))

plt.scatter(X,
            y,
            color='red',
            s=80,
            label='Actual')

plt.scatter(X,
            prediction,
            color='blue',
            s=80,
            label='Prediction')

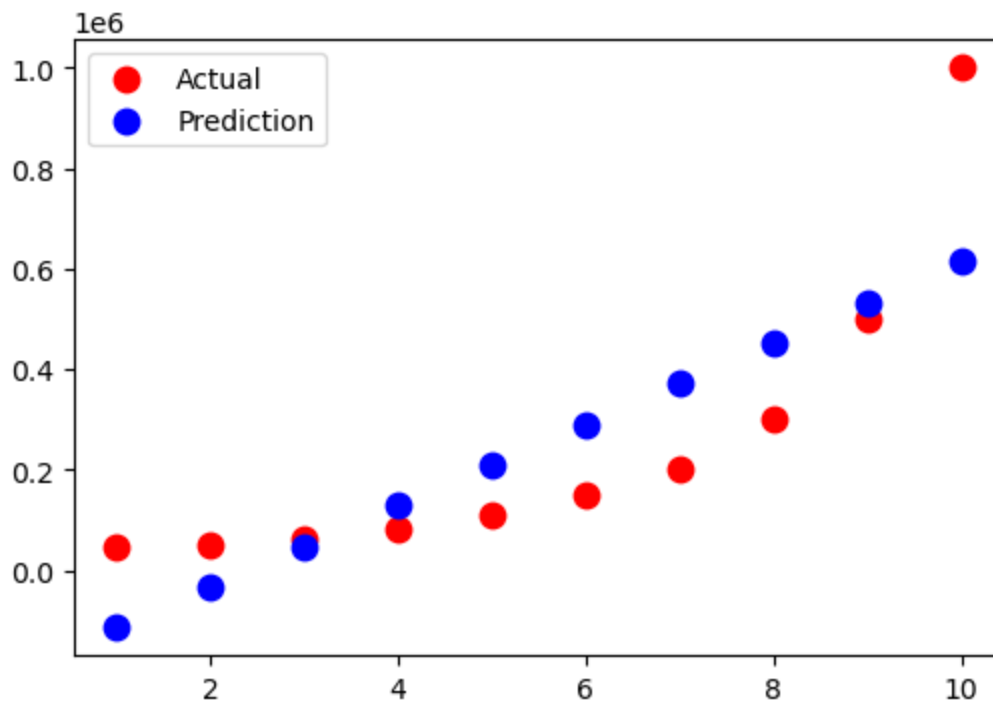
plt.legend()

```

```
plt.show()

# Blue dots = model prediction.

# Red dots = actual data.
```



Method 3 : Show Residual Errors :

```
In [9]: prediction = lin_reg.predict(X)

plt.figure(figsize=(7,5))

plt.scatter(X,
            y,
            color='red')

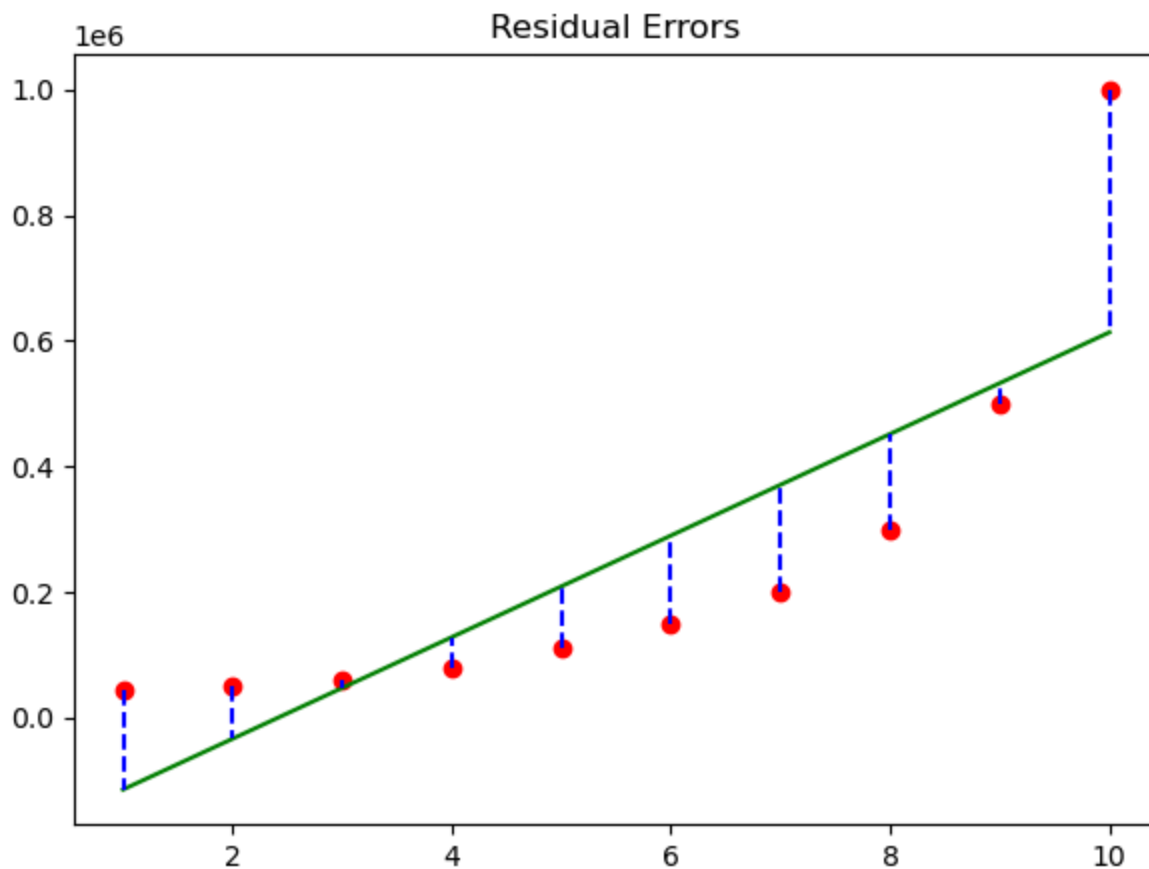
plt.plot(X,
         prediction,
         color='green')

for i in range(len(X)):
    plt.vlines(X[i],
               y[i],
               prediction[i],
               color='blue',
               linestyle='dashed')

plt.title("Residual Errors")

plt.show()
```

```
# Each blue vertical line = error (residual)
```



Method 4 : Predict New Value

```
In [10]: salary = lin_reg.predict([[6.5]])
```

```
plt.figure(figsize=(6,4))
```

```
plt.scatter(X,  
            y,  
            color='red')
```

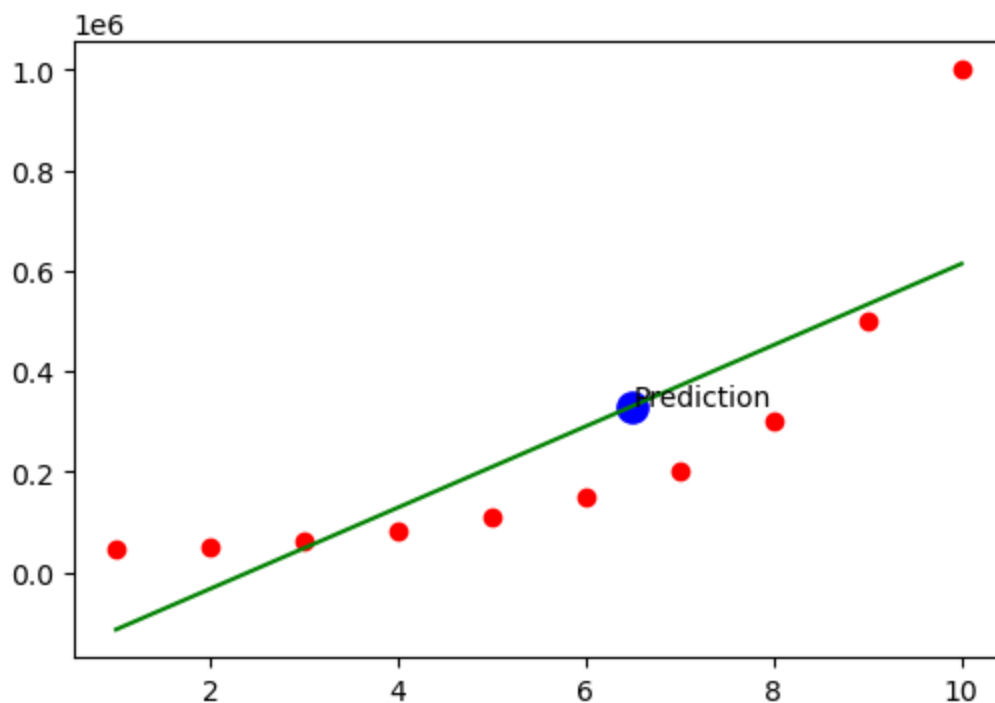
```
plt.plot(X,  
         lin_reg.predict(X),  
         color='green')
```

```
plt.scatter(6.5,  
            salary,  
            color='blue',  
            s=120)
```

```
plt.text(6.5,  
         salary,  
         "Prediction")
```

```
plt.show()
```

```
# Shows where the prediction lies on the regression line.
```



Method 5 : Actual vs Predicted

```
In [11]: prediction = lin_reg.predict(X)

plt.figure(figsize=(6,5))

plt.scatter(y,
            prediction)

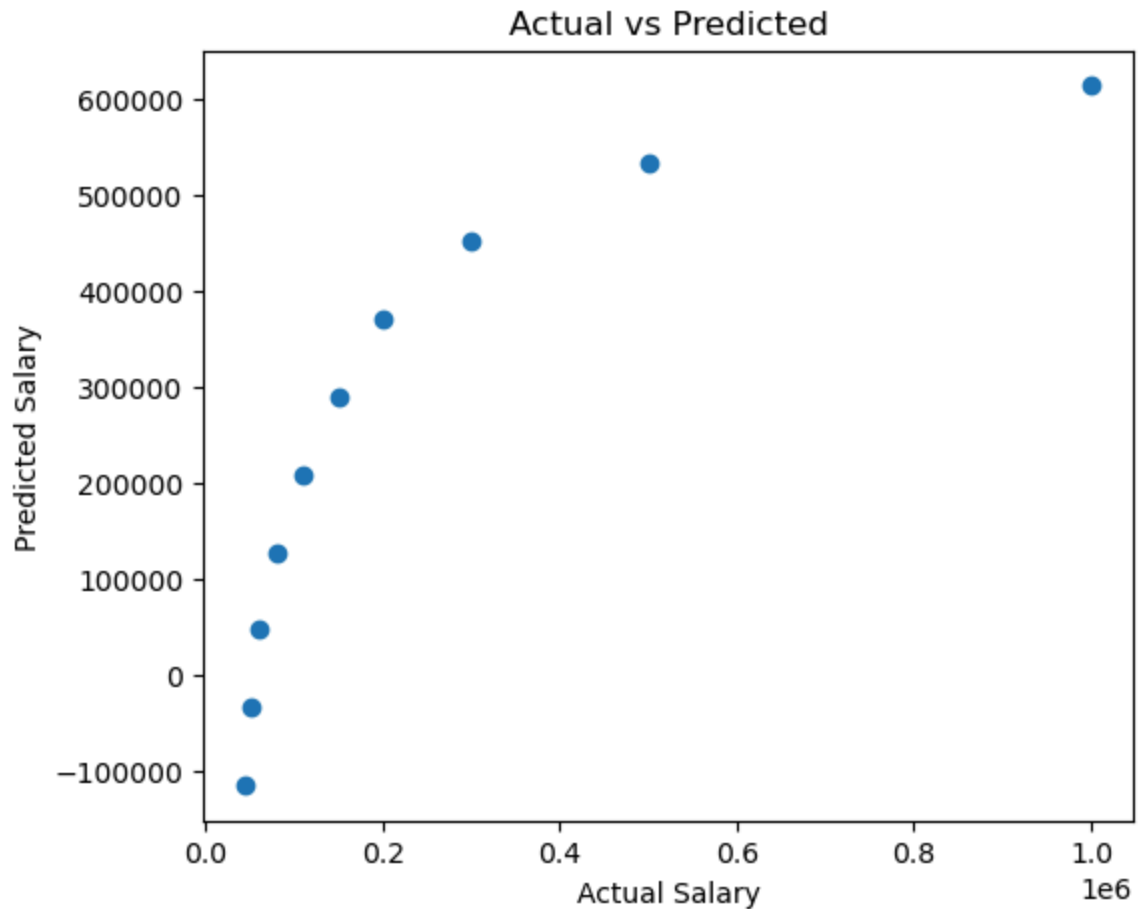
plt.xlabel("Actual Salary")

plt.ylabel("Predicted Salary")

plt.title("Actual vs Predicted")

plt.show()

# If predictions are good, the points will lie close to a diagonal line.
```



Method 6 : Professional visualization

```
In [23]: plt.figure(figsize=(8,5))

plt.scatter(X, y, color='yellow', s = 100, edgecolor='red', label = 'Employeees')

plt.plot(X_grid, lin_reg2.predict(poly.transform(X_grid)),
         color='purple',
         linewidth = 3, label='Polynomial curve')

plt.grid(True)

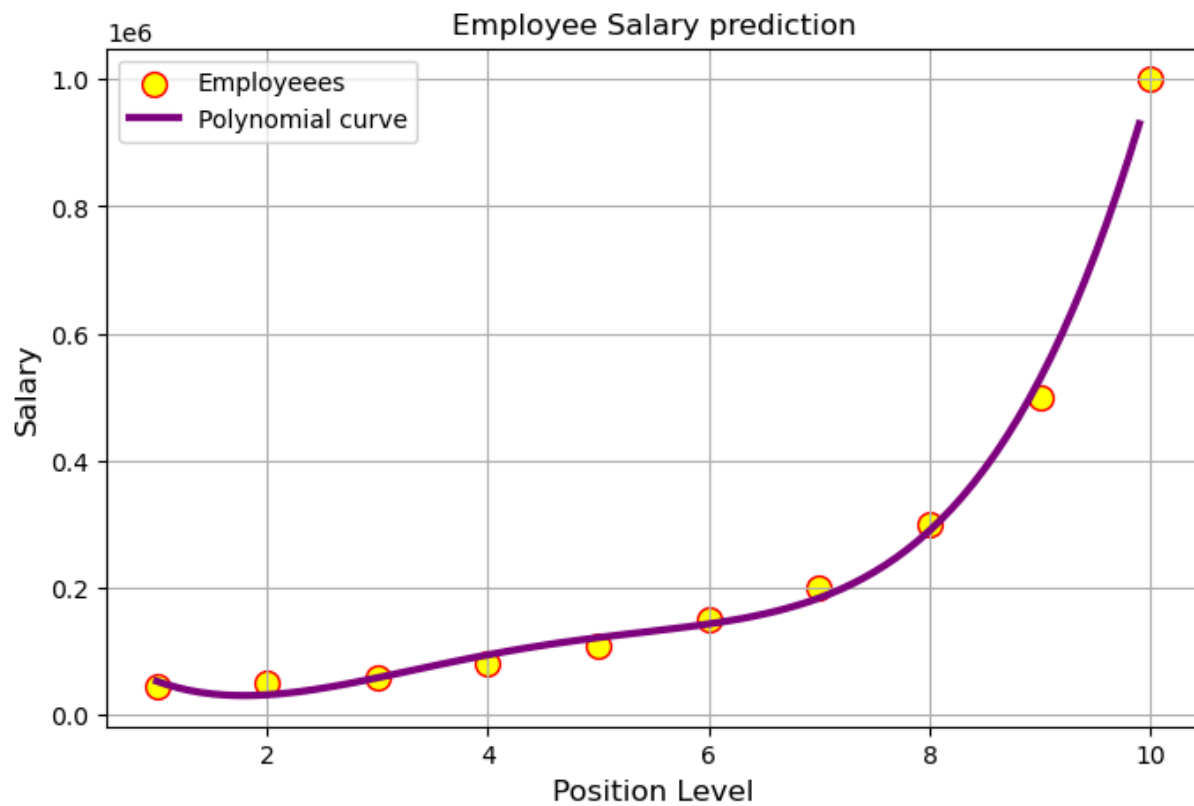
plt.legend()

plt.xlabel("Position Level", fontsize=12)

plt.ylabel("Salary", fontsize=12)

plt.title("Employee Salary prediction")

plt.show()
```



In []: